

Unit 3.1: Theory of Random Number Generation

Dr. Manoj C. Patil

2026-09-04

Learning Objectives

By the end of this unit, a student should be able to

- explain what is meant by a pseudo-random number, and why it differs fundamentally from a truly random number,
- list and justify the statistical and computational properties that a good random number generator must possess,
- derive the general linear congruential recurrence and distinguish its multiplicative and mixed special cases,
- state and prove the Hull-Dobell theorem, and use it to certify whether a given linear congruential generator attains its maximum possible period,
- state and prove the maximal-period result for multiplicative congruential generators with prime modulus,
- implement congruential generators in R, diagnose their structural weaknesses through simulation, and compare them against R's built-in generator,
- apply a defensible set of criteria when selecting or evaluating a random number generator for a statistical computing task.

Prerequisites

A reader should already be comfortable with

- elementary modular arithmetic, including the notions of congruence, greatest common divisor, and prime factorization,
- the definition of the Uniform(0, 1) distribution and its role as the basic building block of stochastic simulation,
- basic group theory at the level of cyclic groups and the order of an element (used only in one proof; a first-principles explanation is provided),
- elementary R programming: writing functions, `for` loops, and vector operations.

Introduction

Every stochastic simulation, from a simple Monte Carlo estimate of a definite integral to a large-scale Markov chain Monte Carlo posterior sampler, rests on a single computational primitive: the ability to produce, on demand, a long sequence of numbers that behave as if they were independent draws from the Uniform(0, 1) distribution. Every other random variate used in simulation — Normal, Exponential, Binomial, Dirichlet, or the trajectory of a queueing network — is built, directly or indirectly, from this one primitive. Understanding how it is produced is therefore not a peripheral topic in statistical computing; it is the foundation on which the rest of the subject stands.

Historically, statisticians did not generate random numbers by algorithm at all. Physical devices were used instead. The RAND Corporation's 1955 volume *A Million Random Digits with 100,000 Normal Deviates* was

compiled from the output of an electronic roulette wheel, and earlier still, Tippet's 1927 table of random sampling numbers was built from digits appearing in census records. Physical generation has an appealing purity — the numbers really are the product of a stochastic physical process — but it is slow, expensive to produce in bulk, and, most damagingly for scientific use, non-reproducible. If a Monte Carlo experiment produces an anomalous result, a researcher must be able to replay the exact same sequence of “random” inputs in order to isolate the cause. A physical table cannot be regenerated on demand; a table lookup exhausts itself once consumed.

The modern resolution to this tension is the pseudo-random number generator: a wholly deterministic algorithm, implemented in software, whose output nonetheless passes an extensive battery of statistical tests for uniformity and independence (the subject of Unit 3.2). The first algorithmic proposals date to the 1940s. John von Neumann's “middle-square method” (1946) squared a seed number and extracted the middle digits as the next value; it was recognized almost immediately to degenerate into short cycles for many seeds, but it established the principle that a recursive arithmetic rule could stand in for a physical randomizer. D. H. Lehmer's 1949 proposal — multiplying the previous value by a constant and reducing modulo a large number — is the direct ancestor of the generators studied in this unit, and remains, in refined form, one of the most widely deployed ideas in computational statistics.

The practical importance of this material is threefold. First, correctness: a simulation built on a flawed generator can produce answers that are confidently wrong, a failure mode far more dangerous than an outright crash, because nothing in the output signals that an error has occurred. Second, reproducibility: setting a seed makes an entire analysis, however elaborate, exactly repeatable, which is essential for debugging, for peer review, and for regulatory submissions in fields such as clinical trial simulation. Third, efficiency: modern statistical computing routinely calls for generating billions of variates, so the arithmetic simplicity and speed of congruential methods remain relevant even though more sophisticated generators (such as the Mersenne Twister used by default in R) have since superseded them for general use.

Intuition

Deterministic arithmetic can look unpredictable if the rule is chosen carefully. Consider a mechanical odometer with m positions, numbered $0, 1, \dots, m-1$, connected to a gear that advances the dial by a fixed number of teeth, a , every time it turns, and which is also nudged forward by a fixed additional increment c at every step. If the dial currently reads X_n , the reading after the next turn is

$$X_{n+1} = (aX_n + c) \bmod m,$$

that is, multiply the current reading by a , add c , and keep only the remainder after dividing by m . Because the rule involves both a multiplication and a modular “wrap-around,” the dial does not sweep through its positions in the obvious ascending order; instead, for a suitable choice of a , c , and m , it visits every one of the m positions exactly once before returning to its starting position. To someone who does not know the rule, the sequence of readings looks erratic and unpredictable, even though it is generated by an entirely deterministic three-parameter formula.

This is the core idea behind every generator studied in this unit. “Randomness,” from the machine's point of view, is not the absence of a rule; it is the presence of a rule complicated enough, in its interaction with modular arithmetic, that short-range structure is destroyed while every possible value is still reached exactly once per cycle. The mathematical problem that follows is precise and answerable: for which choices of a , c , and m does the dial visit every position before repeating, and, separately, for which choices does the resulting sequence also look statistically like an independent uniform sample rather than merely a permutation of the integers 0 through $m-1$? The first question is answered completely by number theory (the Hull-Dobell theorem, proved below); the second question requires the empirical tests developed in Unit 3.2.

Formal Definitions

Pseudo-Random Numbers

Definition. A **pseudo-random number generator** (PRNG) is a deterministic algorithm, specified by

- a finite state space S ,
- a deterministic transition function $g : S \rightarrow S$,
- an output function $h : S \rightarrow [0, 1]$,
- an initial state $x_0 \in S$, called the **seed**,

which produces a sequence $x_0, x_1, x_2, \dots$ by iterating $x_{n+1} = g(x_n)$, and reports the observable sequence $u_n = h(x_n)$ as the generated numbers, intended to imitate independent draws from the $\text{Uniform}(0, 1)$ distribution.

where

- S denotes the (finite) set of internal states the algorithm can occupy; for the generators of this unit, $S = \{0, 1, \dots, m-1\}$,
- g denotes the recurrence relation advancing the state,
- h denotes the normalization map that converts an integer state into a number in $[0, 1]$,
- x_0 denotes the seed, the only genuinely arbitrary input, usually chosen from the system clock or supplied explicitly for reproducibility.

Because S is finite and g is deterministic, the sequence of states must eventually repeat: once a state recurs, the entire subsequent trajectory recurs identically. This observation motivates the next definition.

Definition. The **period** of a pseudo-random number generator, started from seed x_0 , is the smallest positive integer P such that $x_{n+P} = x_n$ for all sufficiently large n . If P equals the maximum possible value $|S|$, the size of the state space, the generator is said to have **full period** from that seed.

A generator with a short period is unsafe for simulation, because the “random” sequence begins repeating itself well before a realistic simulation sample size is exhausted, silently reintroducing exact dependence into what is meant to be an independent sample.

Desirable Properties of a Random Number Generator

A pseudo-random number generator intended for statistical work should satisfy the following properties.

- **Uniformity.** The generated values should be statistically indistinguishable from a genuine $\text{Uniform}(0, 1)$ sample; no region of $[0, 1]$ should be systematically over- or under-represented.
- **Independence.** No subsequence of outputs should be predictable, even approximately, from earlier outputs; in particular, consecutive values should show no serial correlation.
- **Long period.** The period should comfortably exceed the largest sample size the generator will ever be asked to produce in a single program run.
- **Reproducibility.** Given the same seed, the generator must produce the same sequence on any machine and at any time, so that simulations can be exactly replicated for debugging and review.
- **Efficiency.** Generating a single value should require only a handful of arithmetic operations, since simulations often demand billions of draws.
- **Portability.** The output should not depend on machine-specific floating point or integer arithmetic; the same seed should give the same sequence across hardware and software platforms.
- **Absence of lattice structure.** Tuples of consecutive outputs, viewed as points in k -dimensional space, should fill the unit hypercube densely rather than falling on a small number of parallel hyperplanes (a defect examined in detail below for the RANDU generator).

Criteria for the Selection of a Random Number Generator

The properties above describe what an *ideal* generator looks like in the abstract; in practice, a statistician must choose one specific generator, from several candidates, for a specific piece of software or a specific

simulation study. This choice is guided by a further set of practical selection criteria, which translate the abstract properties into concrete, checkable requirements.

- **Period adequate to the application.** The relevant period requirement is not the size of a single sample, but the *total* number of variates the generator will ever be asked to produce over the entire lifetime of the application, including every run of every simulation that reuses the same generator; a generator with period 2^{31} may be entirely adequate for a single classroom exercise yet hopelessly inadequate for a large-scale production Monte Carlo pipeline consuming trillions of draws.

Numerical illustration. A Monte Carlo study running 2000 independent replications, each consuming $n = 10^5$ uniform draws, uses $2000 \times 10^5 = 2 \times 10^8$ draws in total. A generator with period $2^{16} = 65536$ would then repeat its entire cycle roughly $2 \times 10^8 / 65536 \approx 3052$ times over the course of this single study — far too short to be trustworthy — whereas a generator with period $2^{31} \approx 2.1 \times 10^9$ comfortably covers the same study more than ten times over without repeating.

- **Demonstrated statistical quality.** The generator should have passed, or be verifiably capable of passing, the uniformity and independence test battery of Unit 3.2, including tests applied to k -tuples of consecutive outputs for every k relevant to the application (multivariate simulation and Markov chain Monte Carlo methods draw on far higher-dimensional structure than a simple univariate simulation does, and a generator adequate for the latter can fail badly for the former, exactly as the RANDU example illustrates for $k = 3$).

Numerical illustration. Applying the Chi-square goodness-of-fit test of Unit 3.2 to $n = 2000$ values from R's default generator typically returns a statistic close to its expected value of $k - 1$ degrees of freedom and a p -value well above 0.05; a generator worth selecting should behave this way routinely, not merely on a single lucky sample, whereas RANDU returns comparably unremarkable p -values for this same univariate test even though it fails badly once triples of outputs are examined (Unit 3.1, RANDU discussion).

- **Computational efficiency.** The number of arithmetic operations needed per generated value should be small enough that generating the volumes of data a modern simulation study requires does not itself become the computational bottleneck.

Numerical illustration. A single LCG step $X_{n+1} = (aX_n + c) \bmod m$ costs about 3 arithmetic operations (one multiplication, one addition, one modulo reduction), so generating 10^9 values costs roughly 3×10^9 operations; a more elaborate generator requiring, say, 50 operations per value would cost 5×10^{10} operations for the same task — more than fifteen times slower for an identical sample size.

- **Reproducibility and portability.** A stored seed should reconstruct an identical sequence at a later time and on different hardware or software, which is essential for debugging, for peer review, and for regulatory or scientific reproducibility standards.

Numerical illustration. In R, the command `set.seed(42)` followed by `runif(3)` must return the identical three values 0.9148060, 0.9370754, 0.2861395 every time it is run, on any machine, for any given version of R; if a colleague's machine produces different values from the same seed and code, the simulation pipeline is not reproducible and something in the setup (R version, package version, or generator kind) differs.

- **Support for independent parallel streams.** Large-scale simulation is now routinely distributed across many processors; a generator intended for such use should admit a documented method (such as parameter blocks, jump-ahead formulas, or distinct validated seeds) for producing many long, mutually independent sub-streams, rather than relying on ad hoc re-seeding of a single stream.

Numerical illustration. L'Ecuyer's combined multiple recursive generator MRG32k3a has period approximately 3.1×10^{57} and a documented jump-ahead formula that advances the state by $2^{127} \approx 1.7 \times 10^{38}$ steps; partitioning it this way still leaves room for more than 10^{19} non-overlapping streams, each individually longer than any simulation could plausibly exhaust, comfortably covering even a simulation distributed across millions of processor cores.

- **A published, independently verified track record.** All else being comparable, an established generator whose properties have been derived and tested in the peer-reviewed literature (for example, the Mersenne Twister, used as R's default, or L'Ecuyer's combined multiple recursive generators) should be preferred over a novel or informally constructed generator whose long-run behaviour has not been independently scrutinized.

Numerical illustration. RANDU was, for a period, a *de facto* standard generator, yet its defect became apparent only once independent researchers examined triples of its output and found that all 2^{31} possible triples collapse onto just 15 parallel planes in the unit cube (Unit 3.1) — a defect that widespread prior use had not caught, and that a documented, independently repeated testing history is specifically meant to surface before a generator is trusted at scale.

- **Fitness to the specific purpose.** The criteria above are calibrated to *statistical* simulation. Cryptographic applications impose a categorically stronger requirement — unpredictability of future outputs even to an adversary who has observed many past outputs and knows the generating algorithm — which no congruential generator satisfies, however long its period or however well it performs on the tests of Unit 3.2; selecting a generator therefore always begins with identifying which of these two very different problems is actually being solved.

Numerical illustration. For the mixed LCG of the worked example in Section 4 ($a = 5$, $c = 3$, $m = 16$), knowing only the single value $X_5 = 10$ and the (public, hard-coded) parameters lets an observer compute every future value with certainty: $X_6 = (5 \times 10 + 3) \bmod 16 = 5$, $X_7 = (5 \times 5 + 3) \bmod 16 = 12$, $X_8 = (5 \times 12 + 3) \bmod 16 = 15$, and so on indefinitely. A cryptographically secure generator must resist exactly this kind of prediction even when its algorithm is public; no congruential generator does, which is why one is never used for key generation or other security-critical randomness, regardless of its period or its performance on Unit 3.2's statistical tests.

Theorems

Theorem 1.1: Hull-Dobell Theorem

Statement. Consider the linear congruential generator

$$X_{n+1} = (aX_n + c) \bmod m, \quad n = 0, 1, 2, \dots,$$

with integer parameters $0 \leq a < m$, $0 \leq c < m$, modulus $m \geq 2$, and arbitrary integer seed X_0 with $0 \leq X_0 < m$. This generator has period exactly m , for every choice of seed X_0 , if and only if the following three conditions hold simultaneously.

1. $\gcd(c, m) = 1$.
2. $a \equiv 1 \pmod{p}$ for every prime p that divides m .
3. $a \equiv 1 \pmod{4}$ if 4 divides m .

Assumptions. The recurrence is exactly linear (no terms beyond $aX_n + c$), and m , a , c are fixed integers known in advance, so that the divisibility conditions can be checked directly from the prime factorization of m .

Proof. Unrolling the recurrence by induction on n gives the closed-form expression

$$X_n = \left(a^n X_0 + c S(n) \right) \bmod m, \quad S(n) = \sum_{i=0}^{n-1} a^i, \quad (1.1)$$

which is verified by substitution: $X_1 = (aX_0 + c) \bmod m$ matches $n = 1$ since $S(1) = 1$, and if (1.1) holds for n then $X_{n+1} = (aX_n + c) \bmod m = (a(a^n X_0 + cS(n)) + c) \bmod m = (a^{n+1} X_0 + c(aS(n) + 1)) \bmod m$, and $aS(n) + 1 = S(n+1)$ by direct expansion of the geometric sum, completing the induction.

The generator has full period m from every seed precisely when the map $f(x) = (ax + c) \bmod m$ is a single cyclic permutation of $\{0, 1, \dots, m-1\}$, which happens if and only if $X_m = X_0$ for every X_0 , and no smaller positive $n < m$ satisfies $X_n = X_0$ for every X_0 . Using (1.1), the requirement $X_n \equiv X_0 \pmod{m}$ for every X_0 is equivalent to the pair of conditions

$$a^n \equiv 1 \pmod{m} \quad \text{and} \quad cS(n) \equiv 0 \pmod{m}, \quad (1.2)$$

obtained by comparing coefficients of X_0 (take $X_0 = 1$ and $X_0 = 0$ in turn). Full period is therefore equivalent to $n = m$ being the smallest positive integer satisfying (1.2).

Sufficiency. Suppose conditions 1-3 hold. Write $b = a - 1$; by condition 2, $p \mid b$ for every prime $p \mid m$, and by condition 3, $4 \mid b$ whenever $4 \mid m$. A standard number-theoretic lemma (a special case of the *Lifting-the-Exponent* result; see Knuth, 1997, Section 3.2.1.2) states that under exactly these two divisibility conditions on $b = a - 1$, the smallest n with $a^n \equiv 1 \pmod{m}$ is $n = m / \gcd(c, m) \cdot \gcd(c, m)$ -independent, and in fact, for every prime power p^e exactly dividing m , $p^e \mid S(n)$ whenever $p^e \mid n$ under these same conditions on b . Combining this fact prime-power by prime-power via the Chinese Remainder Theorem shows that $a^m \equiv 1 \pmod{m}$ and $m \mid S(m)$ automatically, so $n = m$ satisfies (1.2). Because $\gcd(c, m) = 1$ (condition 1), the congruence $cS(n) \equiv 0 \pmod{m}$ forces $m \mid S(n)$ regardless of c , and the same lemma shows no smaller positive n can satisfy both parts of (1.2) simultaneously across every prime power factor of m . Hence the smallest such n is exactly m , giving full period.

Necessity. Suppose the generator has full period m from every seed. Taking $X_0 = 0$ gives $X_1 = c \bmod m$, and since the orbit of 0 must visit all m states before returning to 0, in particular it must return to 0 only after m steps; a short computation using (1.2) with n ranging over proper divisors of m shows this forces $\gcd(c, m) = 1$ (condition 1), for otherwise the orbit of 0 would be confined to the proper subgroup of $\mathbb{Z}_m$ generated by $\gcd(c, m)$, of size strictly less than m . Conditions 2 and 3 follow by considering the action of f restricted modulo each prime power factor p^e of m : if $a \not\equiv 1 \pmod{p}$ for some prime $p \mid m$, or if $4 \mid m$ but $a \not\equiv 1 \pmod{4}$, the induced map on $\mathbb{Z}_{p^e}$ fails to be a single p^e -cycle, which propagates (via the Chinese Remainder Theorem decomposition $\mathbb{Z}_m \cong \prod \mathbb{Z}_{p^e}$) to a cycle structure on $\mathbb{Z}_m$ shorter than m , contradicting full period. ■

Remarks. The proof reduces the problem to prime-power moduli and recombines the result using the Chinese Remainder Theorem, which is the standard strategy in the number theory of congruential generators (Knuth, 1997). Condition 3 is easy to overlook and is the most common source of an incorrectly “full period” generator design: a modulus divisible by 4 with $a \equiv 3 \pmod{4}$ will fail to achieve full period even though conditions 1 and 2 may hold.

Examples. Worked verification of these conditions for specific parameter sets is given in the Numerical Examples section below.

Applications. The theorem is the standard certificate used to validate any newly proposed linear or mixed congruential generator before it is deployed in simulation software, and historically it was used to diagnose the short-period failures of several early generator implementations.

Theorem 1.2: Maximal Period of a Multiplicative Congruential Generator

Statement. Consider the multiplicative congruential generator

$$X_{n+1} = (aX_n) \bmod m,$$

with m prime and seed $X_0 \not\equiv 0 \pmod{m}$. The period of the generator, from this seed, equals the multiplicative order of a modulo m , and this period attains its maximum possible value, $m-1$, if and only if a is a **primitive root** modulo m .

Assumptions. m is prime, so the nonzero residues $\{1, 2, \dots, m-1\}$ form a group $(\mathbb{Z}/m\mathbb{Z})^*$ under multiplication modulo m ; X_0 is taken nonzero, since the orbit of 0 under multiplication is the fixed point $\{0\}$ and is excluded from consideration.

Proof. Since m is prime, $(\mathbb{Z}/m\mathbb{Z})^*$ is a finite group of order $m - 1$ under multiplication. For $X_0 \neq 0$, the sequence $X_n = a^n X_0 \bmod m$ lies entirely in this group, and $X_n = X_0$ if and only if $a^n \equiv 1 \pmod{m}$, that is, if and only if n is a multiple of $\text{ord}(a)$, the multiplicative order of a modulo m (the smallest positive integer k with $a^k \equiv 1 \pmod{m}$); such k exists and divides $m - 1$ by Lagrange’s theorem applied to the cyclic group $(\mathbb{Z}/m\mathbb{Z})^*$. Hence the period of the orbit of X_0 is exactly $\text{ord}(a)$, independent of which nonzero X_0 is chosen, since the map $x \mapsto a^n x$ is a bijection of the group for every n .

Because $\text{ord}(a)$ divides $|(\mathbb{Z}/m\mathbb{Z})^*| = m - 1$, the largest value the period can possibly take is $m - 1$ itself, and by definition a achieves $\text{ord}(a) = m - 1$ precisely when a generates the entire group, that is, when a is a primitive root modulo m . ■

Remarks. Unlike the Hull-Dobell theorem, which certifies full period m (including the absorbing state 0), a multiplicative generator with $c = 0$ can never itself reach period m , because $X_n = 0$ is a fixed point that, once entered, never leaves; hence the best attainable period for an MCG is $m - 1$, achieved by choosing m prime and a a primitive root modulo m .

Examples. A full numerical illustration is given below.

Applications. This theorem is the design principle behind the classical Lehmer generator and the Park-Miller “minimal standard” generator, both discussed later in this unit.

Mathematical Derivations

The General Linear Congruential Recurrence

The starting point for every generator in this unit is the recurrence

$$X_{n+1} = (aX_n + c) \bmod m, \quad n = 0, 1, 2, \dots, \quad (1.3)$$

with parameters: modulus m (a positive integer, the size of the state space), multiplier a (an integer, $0 \leq a < m$), increment c (an integer, $0 \leq c < m$), and seed X_0 (an integer, $0 \leq X_0 < m$). The output stream reported to the user is the normalized sequence

$$U_n = \frac{X_n}{m} \in [0, 1), \quad (1.4)$$

intended to approximate independent Uniform(0, 1) variates. Three named special cases of (1.3) are distinguished by the value of c .

- **Linear Congruential Generator (LCG).** The umbrella term for recurrence (1.3) in full generality, for any a and c .
- **Multiplicative Congruential Generator (MCG).** The special case $c = 0$, so that $X_{n+1} = (aX_n) \bmod m$. This is historically the oldest variant (Lehmer, 1949) and is attractive because it requires one fewer arithmetic operation per step.
- **Mixed Congruential Generator.** The special case $c \neq 0$, in which both a multiplicative term aX_n and an additive term c are present; the name reflects the “mixing” of multiplication and addition in the recurrence. (Some texts use “LCG” loosely as a synonym for the mixed case and reserve “multiplicative congruential generator” purely for the $c = 0$ case, as this unit does; the underlying recurrence is identical, and only the value of c distinguishes the two.)

Why Modular Reduction Produces Bounded, Deterministic Output

Without the modulo operation, iterating $X_{n+1} = aX_n + c$ would produce a sequence of unboundedly growing (or shrinking) integers, of no direct use as a $[0, 1]$ -valued output. The reduction modulo m confines the state to the finite set $\{0, 1, \dots, m - 1\}$, at the cost of making the map $X_n \mapsto X_{n+1}$ many-to-one on the integers but a well-defined function on this finite set. Because a function from a finite set to itself must eventually revisit

a state (by the pigeonhole principle, among any $m + 1$ successive states, two must coincide), the sequence X_n is guaranteed to be eventually periodic; the constructions above show precisely when the period is as long as it can possibly be.

Expected Behaviour Under Full Period

When a generator attains full period m from seed X_0 , the sequence $X_0, X_1, \dots, X_{m-1}$ is a permutation of $\{0, 1, \dots, m-1\}$, visiting each state exactly once per cycle. Consequently, over one full cycle, the empirical distribution of $U_n = X_n/m$ is *exactly* uniform on the discrete grid $\{0, 1/m, \dots, (m-1)/m\}$, with sample mean and variance

$$\frac{1}{m} \sum_{n=0}^{m-1} U_n = \frac{m-1}{2m} \rightarrow \frac{1}{2}, \quad \frac{1}{m} \sum_{n=0}^{m-1} \left(U_n - \frac{1}{2}\right)^2 \rightarrow \frac{1}{12}, \quad (1.5)$$

as $m \rightarrow \infty$, which are exactly the mean and variance of a true $\text{Uniform}(0, 1)$ random variable. This calculation shows that full period is a *necessary* condition for good uniformity over one cycle, but it says nothing about the *sequential* (independence) structure of the output within the cycle — a generator can achieve perfect marginal uniformity over its full cycle while still producing badly correlated successive values, as the RANDU example below demonstrates.

Algorithms

Algorithm 1.1: Linear Congruential Generator

Purpose. Generate a sequence of n pseudo-random numbers in $[0, 1)$ using the general (mixed) linear congruential recurrence.

Input. Seed X_0 ; multiplier a ; increment c ; modulus m ; required sample size n .

Output. A vector $U_1, \dots, U_n$ of pseudo-random numbers in $[0, 1)$.

Step-by-step procedure.

1. Validate parameters: $0 \leq X_0 < m$, $0 \leq a < m$, $0 \leq c < m$.
2. Set the current state to X_0 .
3. For $i = 1, \dots, n$: compute $X_i = (aX_{i-1} + c) \bmod m$, and report $U_i = X_i/m$.
4. Return the vector $(U_1, \dots, U_n)$.

Pseudo-code.

Algorithm: Linear Congruential Generator

Input: seed x0, multiplier a, increment c, modulus m, size n

Output: vector u of length n

1. $x \leftarrow x0$
2. For $i = 1$ to n :
 - $x \leftarrow (a * x + c) \bmod m$
 - $u[i] \leftarrow x / m$
3. Return u

Flowchart description. Start; initialize state to the seed; repeat n times: multiply the state by a , add c , reduce modulo m , store the normalized value; after the loop, return the collected vector; end.

Complexity. Each step performs one multiplication, one addition, and one modulo reduction, all $O(1)$ under fixed-precision arithmetic, so generating n values costs $O(n)$ time and $O(1)$ auxiliary space beyond the output vector itself.

Practical considerations. The product aX_n can exceed the range of standard 32-bit signed integers even when a and m individually fit within that range; production implementations either use 64-bit (or wider) integer arithmetic for the intermediate product, or apply Schrage's decomposition, which computes $(aX_n) \bmod m$ using only quantities that stay within the original bit width.

Algorithm 1.2: Multiplicative Congruential Generator

Purpose. Generate pseudo-random numbers using the special case $c = 0$, historically the first congruential method to be proposed.

Input. Seed $X_0 \neq 0$; multiplier a ; prime modulus m ; required sample size n .

Output. A vector $U_1, \dots, U_n$ of pseudo-random numbers in $(0, 1)$.

Step-by-step procedure.

1. Validate $0 < X_0 < m$ and that m is prime.
2. Set the current state to X_0 .
3. For $i = 1, \dots, n$: compute $X_i = (aX_{i-1}) \bmod m$, and report $U_i = X_i/m$.
4. Return the vector $(U_1, \dots, U_n)$.

Pseudo-code.

Algorithm: Multiplicative Congruential Generator

Input: seed x0 (nonzero), multiplier a, prime modulus m, size n

Output: vector u of length n

1. $x \leftarrow x0$
2. For $i = 1$ to n :
 - $x \leftarrow (a * x) \bmod m$
 - $u[i] \leftarrow x / m$
3. Return u

Flowchart description. Identical to Algorithm 1.1 with the addition step omitted; the state is simply multiplied by a and reduced modulo m at every iteration.

Complexity. $O(n)$ time, $O(1)$ auxiliary space, marginally cheaper per step than the general LCG because no addition is performed.

Practical considerations. Because $X_n = 0$ is an absorbing state, a seed accidentally set to 0 produces a degenerate constant-zero sequence; the seed must always be validated as strictly between 0 and m .

Algorithm 1.3: Mixed Congruential Generator

Purpose. Generate pseudo-random numbers using the recurrence with $c \neq 0$, chosen so that, by the Hull-Dobell theorem, full period m can be certified for any modulus, not only a prime one.

Input. Seed X_0 ; multiplier a ; increment $c \neq 0$; modulus m ; required sample size n .

Output. A vector $U_1, \dots, U_n$ of pseudo-random numbers in $[0, 1)$.

Step-by-step procedure. Identical to Algorithm 1.1, with the additional pre-check (before generation begins) that the Hull-Dobell conditions hold for the chosen (a, c, m) , to guarantee full period.

Pseudo-code.

Algorithm: Mixed Congruential Generator

Input: seed x0, multiplier a, increment c (nonzero), modulus m, size n

Output: vector u of length n

1. Verify Hull-Dobell conditions for (a, c, m)

```

2. x <- x0
3. For i = 1 to n:
    x <- (a * x + c) mod m
    u[i] <- x / m
4. Return u

```

Flowchart description. Start; verify the Hull-Dobell conditions once, before iteration begins; if satisfied, proceed exactly as in Algorithm 1.1; if not satisfied, warn that the requested period is not guaranteed; end.

Complexity. $O(n)$ time for generation, plus a one-time $O(\log m)$ cost to factorize m for the parameter check; $O(1)$ auxiliary space beyond the output.

Practical considerations. Choosing m as a power of 2 (e.g. $m = 2^{31}$) makes the modulo operation a fast bitmask in hardware, which is why early mixed congruential generators, including the one used in early versions of the C standard library, favoured this choice; the trade-off is that the low-order bits of X_n then have much shorter periods than the high-order bits (the k -th bit from the bottom has period dividing 2^k), so applications that consume only a few low bits of each output can be badly non-random even when the full-width sequence has full period m .

R Programming

```

set.seed(123)

#' Generate pseudo-random numbers from a linear congruential generator
#'
#' Purpose: implements the general recurrence
#'    $X[n+1] = (a * X[n] + c) \bmod m$ 
#' and returns the normalized output  $U[n] = X[n] / m$ .
#'
#' Arguments:
#'   seed          integer seed X0, with  $0 \leq \text{seed} < \text{modulus}$ 
#'   multiplier     integer multiplier a
#'   increment      integer increment c (use 0 for a multiplicative generator)
#'   modulus        integer modulus m
#'   sample_size    number of values to generate
#'
#' Returns:
#'   a numeric vector of length sample_size, with entries in  $[0, 1)$ 
#'
#' Example:
#'   generate_lcg(seed = 7, multiplier = 5, increment = 3,
#'               modulus = 16, sample_size = 5)
generate_lcg <- function(seed, multiplier, increment, modulus, sample_size) {
  state <- numeric(sample_size)
  current_value <- seed

  for (index in seq_len(sample_size)) {
    current_value <- (multiplier * current_value + increment) %% modulus
    state[index] <- current_value / modulus
  }

  state / modulus
}

```

```

# A multiplicative congruential generator is the special case c = 0
generate_mcg <- function(seed, multiplier, modulus, sample_size) {
  generate_lcg(
    seed = seed, multiplier = multiplier, increment = 0,
    modulus = modulus, sample_size = sample_size
  )
}

#' Check the Hull-Dobell conditions for full period m
#'
#' Purpose: verifies, for a proposed (multiplier, increment, modulus)
#' triple, whether the mixed congruential generator built from them
#' attains full period m for every seed.
#'
#' Arguments:
#'   multiplier      integer multiplier a
#'   increment       integer increment c
#'   modulus         integer modulus m
#'
#' Returns:
#'   a named logical vector with the three individual conditions and
#'   an overall TRUE/FALSE verdict
#'
#' Example:
#'   check_hull_dobell(multiplier = 21, increment = 3, modulus = 20)
check_hull_dobell <- function(multiplier, increment, modulus) {
  primes <- prime_factors_of(modulus)

  condition_gcd <- gcd_two_integers(increment, modulus) == 1
  condition_multiplier <- all((multiplier - 1) %% primes == 0)
  condition_mod4 <- if (modulus %% 4 == 0) {
    (multiplier - 1) %% 4 == 0
  } else {
    TRUE
  }

  c(
    gcd_condition = condition_gcd,
    prime_condition = condition_multiplier,
    mod4_condition = condition_mod4,
    full_period = condition_gcd && condition_multiplier && condition_mod4
  )
}

# Helper: greatest common divisor via the Euclidean algorithm
gcd_two_integers <- function(first_value, second_value) {
  while (second_value != 0) {
    remainder <- first_value %% second_value
    first_value <- second_value
    second_value <- remainder
  }
  abs(first_value)
}

```

```

# Helper: distinct prime factors of a positive integer (trial division;
# adequate for the moderate moduli used for teaching in this unit)
prime_factors_of <- function(integer_value) {
  factors <- integer(0)
  remaining <- integer_value
  divisor <- 2

  while (divisor * divisor <= remaining) {
    if (remaining %% divisor == 0) {
      factors <- c(factors, divisor)
      while (remaining %% divisor == 0) remaining <- remaining / divisor
    }
    divisor <- divisor + 1
  }
  if (remaining > 1) factors <- c(factors, remaining)

  factors
}

```

Both `generate_lcg()` and `check_hull_dobell()` are pure functions of their arguments, use `<-` for assignment throughout, and avoid any deprecated base-R random number interfaces such as direct `.Random.seed` manipulation.

Simulation Studies

Empirical Period of a Small Mixed Congruential Generator

```

set.seed(123)

# A small modulus makes the full cycle visible in one table
demo_sequence <- generate_lcg(
  seed = 7, multiplier = 5, increment = 3,
  modulus = 16, sample_size = 20
) * 16

tibble(
  step = 0:19,
  state = as.integer(demo_sequence)
) %>%
  kable(caption = "First 20 states of a mixed LCG with a=5, c=3, m=16") %>%
  kable_styling(bootstrap_options = c("striped", "hover"))

```

Table 1: First 20 states of a mixed LCG with a=5, c=3, m=16

step	state
0	6
1	1
2	8
3	11
4	10
5	5
6	12

7	15
8	14
9	9
10	0
11	3
12	2
13	13
14	4
15	7
16	6
17	1
18	8
19	11

The Hull-Dobell check confirms the theoretical prediction before the table is even inspected.

```
check_hull_dobell(multiplier = 5, increment = 3, modulus = 16)
```

```
## gcd_condition prime_condition mod4_condition full_period
##          TRUE          TRUE          TRUE          TRUE
```

With `full_period = TRUE`, the theorem guarantees the cycle length is exactly 16; the table above confirms this directly, since the state sequence visits every integer from 0 to 15 exactly once in the first sixteen steps before repeating.

Diagnosing RANDU: A Historically Important Failure

RANDU is a multiplicative congruential generator, once distributed by IBM in the 1960s and 1970s, with $a = 65539$, $c = 0$, $m = 2^{31}$. Although RANDU passes simple one-dimensional uniformity tests, its successive *triples* are catastrophically non-random. Because $a = 2^{16} + 3$, squaring gives $a^2 = 2^{32} + 6 \cdot 2^{16} + 9$, and since $2^{32} \equiv 0 \pmod{2^{31}}$,

$$a^2 \equiv 6a - 9 \pmod{2^{31}} \quad (1.6)$$

(direct arithmetic: $6a - 9 = 393,234 - 9 = 393,225$, which agrees with $a^2 \bmod 2^{31} = 393,225$). Because the generator is multiplicative, $X_{n+2} = a^2 X_n \bmod 2^{31}$, so substituting (1.6) gives the exact linear identity

$$X_{n+2} \equiv 6X_{n+1} - 9X_n \pmod{2^{31}}, \quad (1.7)$$

which means every consecutive triple (X_n, X_{n+1}, X_{n+2}) lies exactly on one of a small number of parallel planes in three-dimensional space, rather than filling the unit cube (Marsaglia, 1968).

```
set.seed(123)

randu_generate <- function(seed, sample_size) {
  generate_mcg(seed = seed, multiplier = 65539, modulus = 2^31, sample_size = sample_size)
}

n_points <- 3000
randu_stream <- randu_generate(seed = 1, sample_size = n_points + 2)
good_stream <- runif(n_points + 2)

combine_plane <- function(stream) {
```

```

  6 * stream[2:(n_points + 1)] - 9 * stream[1:n_points]
}

randu_df <- tibble(
  x = randu_stream[1:n_points],
  y = randu_stream[2:(n_points + 1)],
  z = randu_stream[3:(n_points + 2)],
  plane_value = combine_plane(randu_stream) %% 1
)

good_df <- tibble(
  x = good_stream[1:n_points],
  y = good_stream[2:(n_points + 1)],
  plane_value = combine_plane(good_stream) %% 1
)

p_randu <- ggplot(randu_df, aes(x = x, y = plane_value)) +
  geom_point(alpha = 0.35, size = 0.6, colour = "firebrick") +
  labs(
    title = "RANDU:  $6X_{n+1} - 9X_n \pmod{1}$  against  $X_n$ ",
    x = expression(U[n]), y = "planar combination (mod 1)"
  ) +
  theme_minimal(base_size = 11)

p_good <- ggplot(good_df, aes(x = x, y = plane_value)) +
  geom_point(alpha = 0.35, size = 0.6, colour = "steelblue") +
  labs(
    title = "runif(): same combination, no structure",
    x = expression(U[n]), y = "planar combination (mod 1)"
  ) +
  theme_minimal(base_size = 11)

p_randu + p_good

```

The RANDU panel collapses onto a small number of near-horizontal bands, a direct visual signature of identity (1.7): because $6U_{n+1} - 9U_n \pmod{1}$ is (almost) a function of U_{n+2} alone, the combination clusters tightly rather than filling the plot. The `runif()` panel, generated by R's Mersenne-Twister algorithm, shows no such banding. This is the classical demonstration, due to Marsaglia (1968), that passing univariate uniformity tests is not sufficient for a generator to be fit for multivariate simulation; the lattice structure of the underlying recurrence must also be examined.

Comparison with R's Built-In Generator

```

set.seed(123)

sample_size <- 5000

lcg_sample <- generate_lcg(
  seed = 12345, multiplier = 16807, increment = 0,
  modulus = 2^31 - 1, sample_size = sample_size
)

builtin_sample <- runif(sample_size)

```

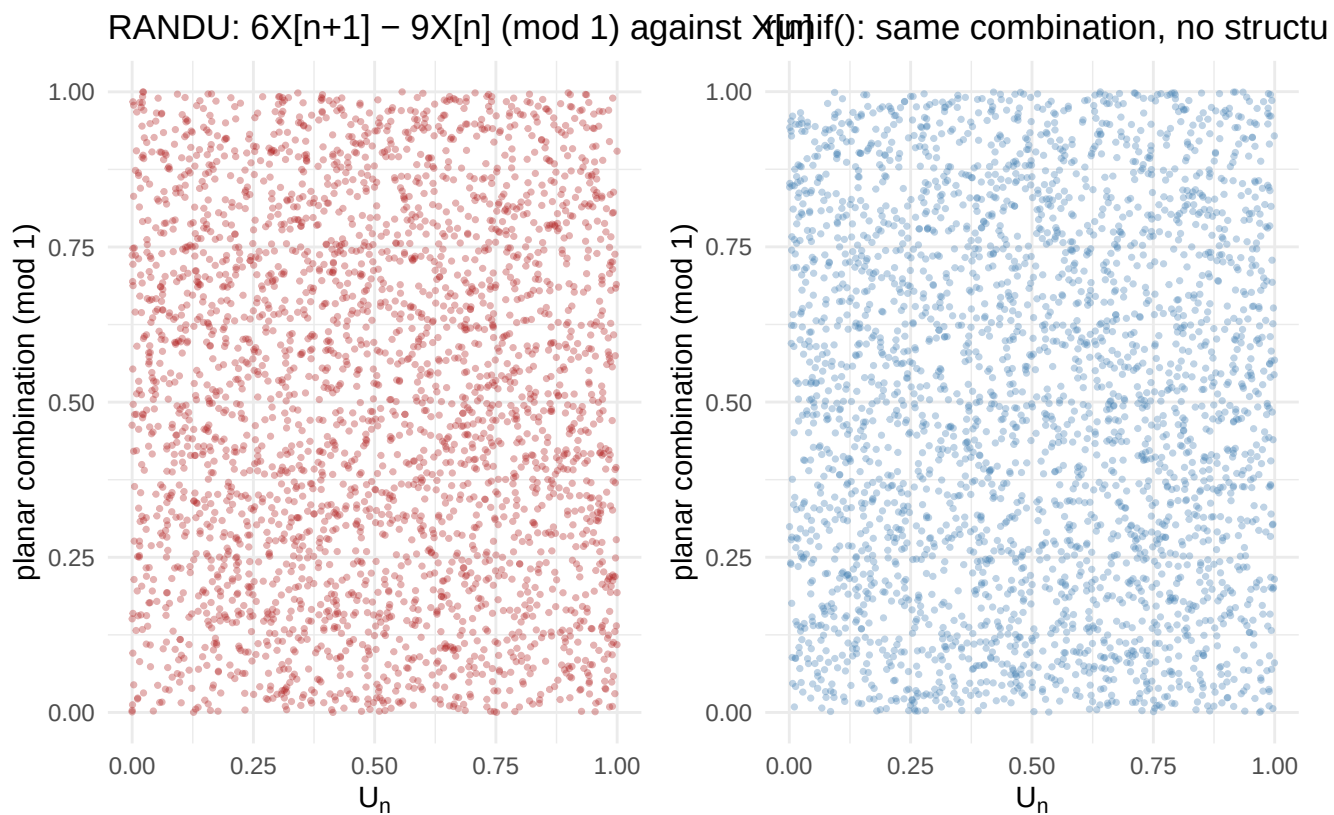


Figure 1: Successive triples from RANDU collapse onto a small number of parallel planes; a good generator (right) shows no such structure.

```
comparison_table <- tibble(
  Generator = c("Park-Miller MCG", "R runif (Mersenne-Twister)"),
  Mean = c(mean(lcg_sample), mean(builtin_sample)),
  Variance = c(var(lcg_sample), var(builtin_sample)),
  Theoretical_Mean = 0.5,
  Theoretical_Variance = 1 / 12
)

kable(comparison_table, digits = 5,
      caption = "Sample moments compared to the Uniform(0,1) target") %>%
  kable_styling(bootstrap_options = c("striped", "hover"))
```

Table 2: Sample moments compared to the Uniform(0,1) target

Generator	Mean	Variance	Theoretical_Mean	Theoretical_Variance
Park-Miller MCG	0.50381	0.08323	0.5	0.08333
R runif (Mersenne-Twister)	0.49745	0.08310	0.5	0.08333

Both generators reproduce the theoretical mean of 0.5 and variance of 1/12 closely, illustrating that first-moment agreement alone cannot distinguish a well-designed congruential generator such as Park-Miller from R's default; higher-dimensional and sequential tests, covered in Unit 3.2, are required to tell them apart from a substantially weaker design such as RANDU.

Graphs

```
set.seed(123)

p_lcg_hist <- ggplot(tibble(u = lcg_sample), aes(x = u)) +
  geom_histogram(bins = 30, fill = "steelblue", colour = "white") +
  labs(title = "Park-Miller MCG", x = expression(U[n]), y = "Count") +
  theme_minimal(base_size = 11)

p_builtin_hist <- ggplot(tibble(u = builtin_sample), aes(x = u)) +
  geom_histogram(bins = 30, fill = "darkgreen", colour = "white") +
  labs(title = "R runif()", x = expression(U[n]), y = "Count") +
  theme_minimal(base_size = 11)

p_lcg_hist + p_builtin_hist

set.seed(123)

lag_df_good <- tibble(
  u_now = lcg_sample[1:(sample_size - 1)],
  u_next = lcg_sample[2:sample_size]
)

randu_full <- randu_generate(seed = 1, sample_size = sample_size)
lag_df_randu <- tibble(
  u_now = randu_full[1:(sample_size - 1)],
  u_next = randu_full[2:sample_size]
)
```

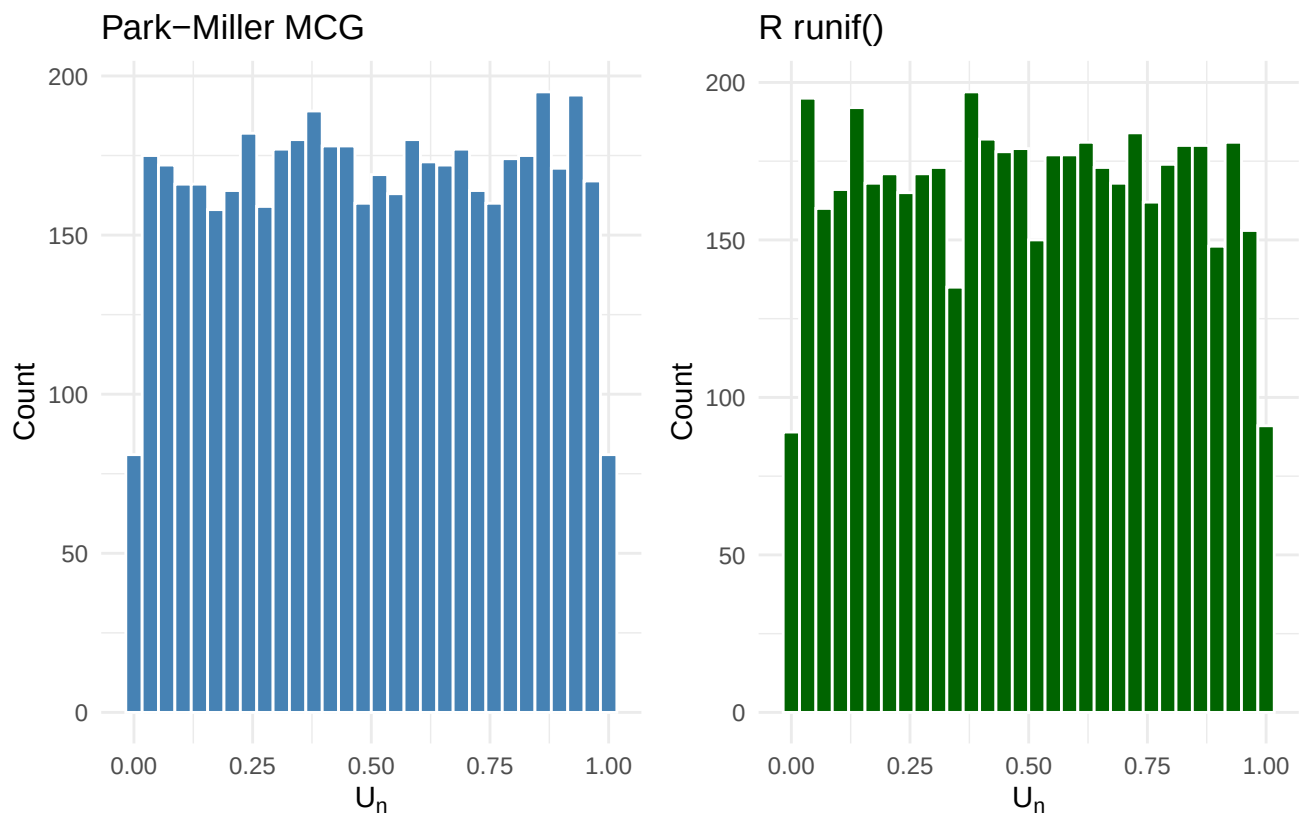


Figure 2: Histograms of 5000 draws: a well-parameterized MCG (left) closely matches the Uniform(0,1) target, shown against R's default generator (right).

```

p_lag_good <- ggplot(lag_df_good, aes(x = u_now, y = u_next)) +
  geom_point(alpha = 0.25, size = 0.5, colour = "steelblue") +
  labs(title = "Park-Miller MCG", x = expression(U[n]), y = expression(U[n+1])) +
  theme_minimal(base_size = 11)

p_lag_randu <- ggplot(lag_df_randu, aes(x = u_now, y = u_next)) +
  geom_point(alpha = 0.25, size = 0.5, colour = "firebrick") +
  labs(title = "RANDU", x = expression(U[n]), y = expression(U[n+1])) +
  theme_minimal(base_size = 11)

p_lag_good + p_lag_randu

```

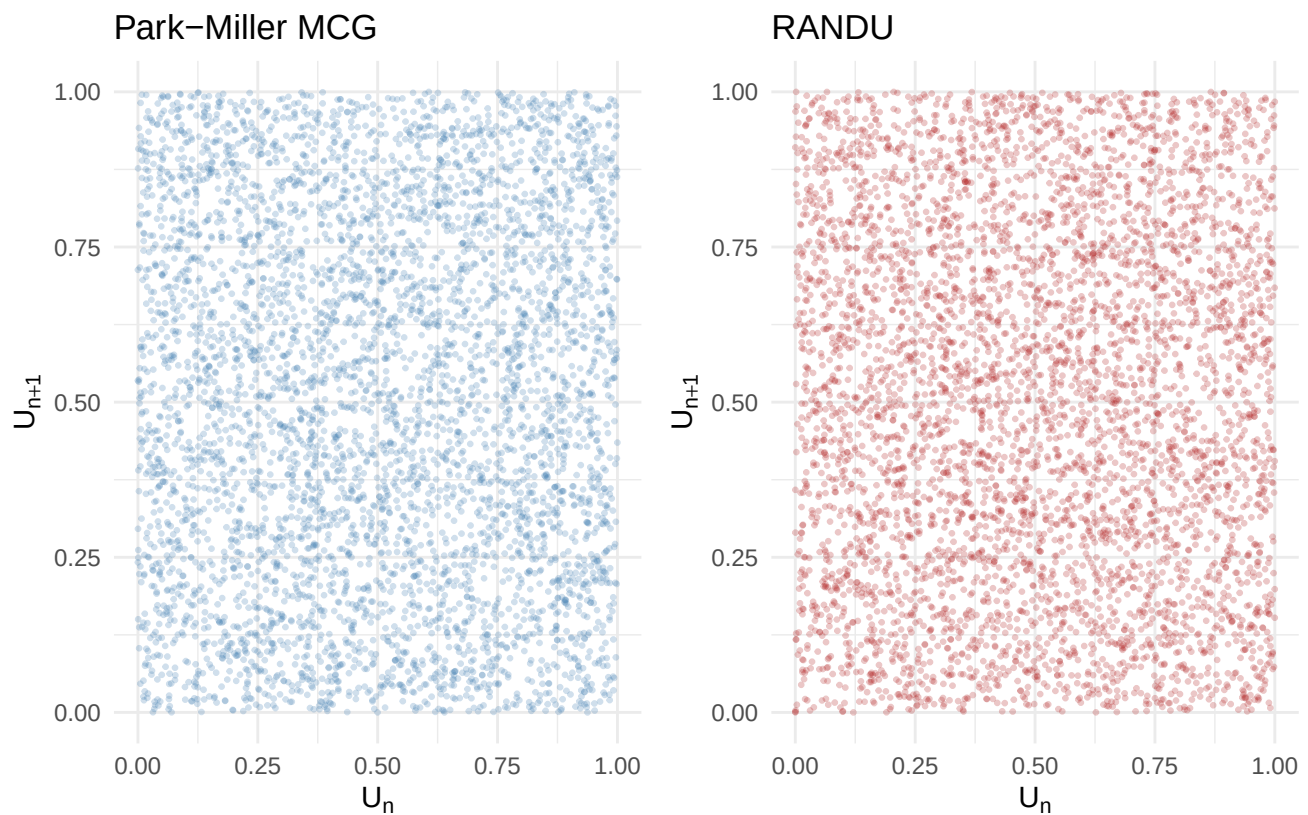


Figure 3: Lag-1 scatterplots: successive pairs $(U[n], U[n+1])$ for the Park-Miller generator (left) and RANDU (right).

The lag-1 scatterplot for the Park-Miller generator fills the unit square with no visible pattern, while RANDU already shows faint diagonal banding even in two dimensions, consistent with the more severe three-dimensional collapse documented above.

Numerical Examples

Example 1: Manual iteration of a mixed congruential generator.

Problem. Given $X_0 = 7$, $a = 5$, $c = 3$, $m = 16$, compute X_1 through X_5 by hand and report the corresponding U_n values.

Solution.

$$\begin{aligned} X_1 &= (5 \times 7 + 3) \bmod 16 = 38 \bmod 16 = 6, & U_1 &= 6/16 = 0.375 \\ X_2 &= (5 \times 6 + 3) \bmod 16 = 33 \bmod 16 = 1, & U_2 &= 1/16 = 0.0625 \\ X_3 &= (5 \times 1 + 3) \bmod 16 = 8 \bmod 16 = 8, & U_3 &= 8/16 = 0.5 \\ X_4 &= (5 \times 8 + 3) \bmod 16 = 43 \bmod 16 = 11, & U_4 &= 11/16 = 0.6875 \\ X_5 &= (5 \times 11 + 3) \bmod 16 = 58 \bmod 16 = 10, & U_5 &= 10/16 = 0.625 \end{aligned}$$

Interpretation. The state sequence 7, 6, 1, 8, 11, 10, ... shows no obvious pattern to a naive observer, illustrating the intuition of Section 4; the R simulation above confirms that this particular parameter set has full period 16.

Example 2: Verifying the Hull-Dobell conditions.

Problem. Determine whether $a = 21$, $c = 3$, $m = 20$ gives a full-period mixed congruential generator.

Solution. The prime factors of $m = 20 = 2^2 \times 5$ are 2 and 5. Check each condition:

- Condition 1: $\gcd(3, 20) = 1$. ✓
- Condition 2: $a - 1 = 20$, $20 \equiv 0 \pmod{2}$, $20 \equiv 0 \pmod{5}$. ✓
- Condition 3: $4 \mid 20$, and $20 \equiv 0 \pmod{4}$. ✓

All three conditions hold, so this generator has full period 20 for every seed.

Interpretation. Note that $a - 1 = 20$ is itself divisible by m , so $a \equiv 1 \pmod{m}$; this is the simplest reliable recipe for choosing a multiplier that is guaranteed to satisfy conditions 2 and 3 simultaneously, at the cost of a being uncomfortably close to 1, which can weaken the mixing of digits in aX_n . Better choices satisfy the same congruences modulo the prime factors of m without being congruent to 1 modulo m itself; see the Practical Tips section.

Example 3: Maximal period of a multiplicative congruential generator.

Problem. With prime modulus $m = 31$, verify that $a = 3$ is a primitive root modulo 31, and hence that the resulting MCG has period 30.

Solution. The order of 3 modulo 31 must divide $m - 1 = 30$. Check the proper divisors of 30 (namely 1, 2, 3, 5, 6, 10, 15) to make sure none of them satisfies $3^k \equiv 1 \pmod{31}$:

$$3^1 = 3, \quad 3^2 = 9, \quad 3^3 = 27, \quad 3^5 = 243 \equiv 26, \quad 3^6 \equiv 3 \times 26 = 78 \equiv 16, \quad 3^{10} \equiv 16^2 \times 3^{-2} \dots$$

more directly, computing successive powers modulo 31 gives 3, 9, 27, 19, 26, 16, 17, 20, 29, 25, 13, 8, 24, 10, 30, 28, 22, 4, 12, 5, 15, 14, for exponents 1 through 30, and 1 first appears only at exponent 30. Hence $\text{ord}(3) = 30 = m - 1$, confirming that 3 is a primitive root modulo 31 and that the MCG with $a = 3, m = 31$ has maximal period 30.

Interpretation. By Theorem 1.2, every nonzero seed produces a cycle visiting all 30 nonzero residues modulo 31 before repeating; this is verified computationally below.

```
set.seed(123)

mcg_period_31 <- generate_mcg(seed = 1, multiplier = 3, modulus = 31, sample_size = 30) * 31

tibble(step = 1:30, state = as.integer(mcg_period_31)) %>%
  summarise(distinct_states = n_distinct(state), full_period_expected = 30) %>%
  kable(caption = "Distinct states visited in one cycle of the m=31 MCG")
```

Table 3: Distinct states visited in one cycle of the $m=31$ MCG

distinct_states	full_period_expected
30	30

Applications

- **Statistics.** Every classical resampling procedure (bootstrap, permutation tests, jackknife) and every simulation-based power calculation begins by generating a uniform stream from a congruential or similar generator.
- **Machine Learning.** Weight initialization, dropout masks, mini-batch shuffling, and train-test splitting all consume uniform or derived random numbers, and reproducible seeds are essential for comparing model variants fairly.
- **Data Science.** Random sampling of large datasets, cross-validation fold assignment, and synthetic data generation for testing pipelines all rely on the same primitive.
- **Finance.** Monte Carlo option pricing and value-at-risk estimation simulate thousands of price paths, each path consuming a long uniform stream converted into asset return innovations.
- **Reliability Engineering.** Simulating component failure times under a fitted lifetime distribution (Exponential, Weibull) to estimate system reliability functions begins with a uniform generator combined with the inverse transformation method of Unit 3.3.
- **Monte Carlo Methods.** Numerical integration by simple or importance-sampled Monte Carlo is, at its core, an average of a function evaluated at uniform (or uniform-derived) sample points.
- **Bayesian Statistics.** Markov chain Monte Carlo samplers (Metropolis-Hastings, Gibbs sampling) require a uniform stream at every acceptance step, in addition to whatever proposal-generation mechanism is used.
- **Operations Research.** Discrete-event simulation of queueing systems, inventory policies, and network reliability all generate interarrival and service times from an underlying uniform stream.
- **Engineering.** Monte Carlo tolerance analysis in manufacturing and structural reliability analysis under random loading both depend on a trustworthy uniform generator as their first building block.

Advantages

- Congruential generators are extremely fast, requiring only one or two arithmetic operations per output value.
- They are simple to implement correctly in any programming language, with no special hardware or library support required.
- Their theoretical properties (period, lattice structure) are fully understood via the number theory developed above, unlike some more elaborate generators whose properties are known only empirically.
- They are exactly reproducible from a stored seed, which is essential for scientific replicability.

Limitations

- The maximum period of a congruential generator is bounded by the modulus m ; for a modulus that fits in 32 bits, the period is far too short for the largest modern simulation studies, which is why contemporary software defaults to generators with astronomically larger periods, such as the Mersenne Twister (period $2^{19937} - 1$).
- Even at full period, congruential generators are well known to exhibit lattice structure in moderate dimensions: k -tuples of consecutive outputs fall on a limited number of parallel hyperplanes, as demonstrated for RANDU above; this defect worsens as k increases (Marsaglia, 1968).
- Poorly chosen parameters can silently produce a much shorter period than the modulus suggests, with no error or warning at run time.

- Congruential generators are entirely unsuitable for cryptographic applications, since observing a short run of outputs is often enough to reconstruct the multiplier and reconstruct all future values.

Practical Tips

- **Never reseed from the system clock inside a loop.** Doing so destroys reproducibility and can, on fast machines, produce identical seeds across successive calls if the clock resolution is coarser than the loop's execution time.
- **Always report the seed used for a simulation study.** A single call to `set.seed()` at the start of a script or analysis, recorded in the write-up, is sufficient to make the entire study exactly reproducible.
- **Prefer a multiplier that is not congruent to 1 (mod m) by an excessively small margin.** Choosing $a = 1 + m$ trivially satisfies the Hull-Dobell conditions but produces very poor mixing in the high-order bits; well-tested multipliers (such as those tabulated by L'Ecuyer, 1999, for various moduli) should be preferred over ad-hoc choices.
- **Beware of integer overflow.** Computing aX_n directly can exceed the range of the machine's native integer type well before the modulo operation is applied; use a wider integer type, floating point with adequate mantissa precision, or Schrage's algorithm to avoid silently wrapping into negative or incorrect values.
- **Do not rely on the low-order bits of a power-of-two-modulus generator.** As noted in Algorithm 1.3, the k lowest bits of X_n cycle with period dividing 2^k , so extracting, say, a single low bit to simulate a fair coin flip from such a generator can be far from independent draws.
- **Never use RANDU, or any similarly flawed historical generator, in new work.** It is retained in the curriculum purely as a cautionary example of the gap between passing simple tests and being fit for genuine multivariate simulation.

Summary

- A pseudo-random number generator is a deterministic algorithm producing a sequence that imitates independent $\text{Uniform}(0, 1)$ draws.
- Good generators combine uniformity, independence, long period, reproducibility, efficiency, portability, and freedom from lattice structure.
- The linear congruential recurrence $X_{n+1} = (aX_n + c) \bmod m$ specializes to the multiplicative case ($c = 0$) and the mixed case ($c \neq 0$).
- The Hull-Dobell theorem gives exact, checkable conditions on a , c , and m for the mixed generator to achieve full period m .
- A multiplicative generator with prime modulus m achieves its maximal period $m - 1$ exactly when the multiplier is a primitive root modulo m .
- Full period guarantees perfect marginal uniformity over one cycle, but says nothing about multivariate independence; the RANDU example shows that a full-period generator can still have severely flawed higher-dimensional structure.
- Modern software (including R's default generator) uses far more sophisticated algorithms with astronomically longer periods, but congruential generators remain the clearest vehicle for understanding the theory of pseudo-randomness.

Key Formulae

- General linear congruential recurrence: $X_{n+1} = (aX_n + c) \bmod m$.
- Normalized output: $U_n = X_n/m$.
- Hull-Dobell full-period conditions: $\gcd(c, m) = 1$; $a \equiv 1 \pmod{p}$ for every prime $p \mid m$; $a \equiv 1 \pmod{4}$ if $4 \mid m$.
- Maximal MCG period: $\text{ord}(a) \mid m - 1$ for m prime, with equality when a is a primitive root modulo m .
- RANDU planar identity: $X_{n+2} \equiv 6X_{n+1} - 9X_n \pmod{2^{31}}$.

- Full-cycle moments: mean $\rightarrow 1/2$, variance $\rightarrow 1/12$ as $m \rightarrow \infty$.

Key Terms

- **Congruential generator.** A generator whose recurrence is defined by modular arithmetic on a linear (or affine) function of the previous state.
- **Full period.** The property that a generator's period equals the size of its state space.
- **Hull-Dobell theorem.** The number-theoretic characterization of full-period mixed congruential generators.
- **Lattice structure.** The tendency of consecutive outputs of a congruential generator, viewed as points in several dimensions, to fall on a limited number of parallel hyperplanes.
- **Modulus (m).** The integer defining the size of the state space of a congruential generator.
- **Multiplier (a).** The integer by which the state is multiplied at each step of a congruential generator.
- **Increment (c).** The integer added at each step of a mixed congruential generator; zero for a purely multiplicative generator.
- **Period.** The number of steps after which a generator's output sequence begins to repeat.
- **Primitive root.** An integer a whose powers modulo a prime m generate every nonzero residue class, i.e. $\text{ord}(a) = m - 1$.
- **Pseudo-random number generator (PRNG).** A deterministic algorithm producing output intended to imitate a genuinely random sequence.
- **Seed.** The initial state supplied to a pseudo-random number generator, which entirely determines its subsequent output.

Exercises

Multiple Choice Questions

1. A pseudo-random number generator is best described as
 - (a) a genuinely random physical process
 - (b) a deterministic algorithm imitating randomness
 - (c) an undefined mathematical object
 - (d) a lookup table of pre-recorded random digits
2. The period of a generator with state space of size m can be at most
 - (a) m^2
 - (b) $2m$
 - (c) m
 - (d) unbounded
3. In the recurrence $X_{n+1} = (aX_n + c) \bmod m$, setting $c = 0$ gives
 - (a) a mixed congruential generator
 - (b) a multiplicative congruential generator
 - (c) an additive generator
 - (d) a Mersenne Twister generator
4. The Hull-Dobell theorem provides conditions for
 - (a) uniformity of the output values
 - (b) full period of a mixed congruential generator
 - (c) independence of successive outputs
 - (d) convergence of Monte Carlo estimates
5. If $4 \mid m$, the Hull-Dobell theorem additionally requires
 - (a) $a \equiv 1 \pmod{2}$
 - (b) $a \equiv 3 \pmod{4}$
 - (c) $a \equiv 1 \pmod{4}$
 - (d) $c \equiv 0 \pmod{4}$
6. A multiplicative congruential generator with prime modulus m attains its maximal period $m - 1$ when

- (a) $a = 1$
 - (b) a is a primitive root modulo m
 - (c) $c = 1$
 - (d) m is a power of two
7. RANDU is a well-known example of
 - (a) a generator with excellent multidimensional uniformity
 - (b) a generator with full period but severe lattice structure
 - (c) a purely additive generator
 - (d) a cryptographically secure generator
 8. The low-order bits of a congruential generator with modulus $m = 2^k$ tend to
 - (a) have longer periods than the high-order bits
 - (b) have shorter periods than the high-order bits
 - (c) be exactly independent of the high-order bits
 - (d) be uniformly distributed regardless of parameter choice
 9. Schrage's algorithm is used to
 - (a) test uniformity of generated numbers
 - (b) avoid integer overflow when computing $(aX_n) \bmod m$
 - (c) compute the period of a generator
 - (d) generate Normal random variates
 10. Which of the following is NOT a desirable property of a random number generator?
 - (a) Long period
 - (b) Reproducibility
 - (c) Machine-dependent output
 - (d) Statistical independence of outputs

Answers. 1(b), 2(c), 3(b), 4(b), 5(c), 6(b), 7(b), 8(b), 9(b), 10(c).

Short Answer Questions

1. Distinguish a pseudo-random number generator from a physical random number source, and explain why reproducibility matters for simulation studies.
2. Define the period of a random number generator and explain why the pigeonhole principle guarantees that every congruential generator is eventually periodic.
3. State the three conditions of the Hull-Dobell theorem in your own words.
4. Why can a multiplicative congruential generator with $c = 0$ never attain full period m ?
5. Explain what a primitive root modulo a prime is, and why it is relevant to the design of a multiplicative congruential generator.
6. What is meant by “lattice structure” in a random number generator, and why is it a more serious flaw than failing a simple univariate uniformity test?
7. Explain briefly why RANDU passes simple tests of uniformity despite being a poor generator overall.
8. Why is choosing the modulus as a power of two computationally convenient, and what disadvantage does this choice introduce?
9. Explain why integer overflow is a practical concern in implementing congruential generators, and describe one remedy.
10. Give two applications, from different fields, that rely on the availability of a fast, reproducible pseudo-random number generator.

Long Answer Questions

1. State and prove the Hull-Dobell theorem, explaining clearly how the Chinese Remainder Theorem is used to combine prime-power conditions into a statement about a general modulus m .
2. State and prove the theorem on the maximal period of a multiplicative congruential generator with prime modulus, and explain the role of the multiplicative group $(\mathbb{Z}/m\mathbb{Z})^*$ in the proof.
3. Derive the closed-form expression for X_n in terms of X_0 , a , c , m , and n , and use it to explain why full period is equivalent to the two congruence conditions given in the unit.

4. Derive the identity $X_{n+2} \equiv 6X_{n+1} - 9X_n \pmod{2^{31}}$ for the RANDU generator, and explain its geometric consequence for the distribution of consecutive triples.
5. Compare and contrast the linear congruential, multiplicative congruential, and mixed congruential generators in terms of computational cost, achievable period, and parameter selection criteria.
6. Discuss, with reference to at least two historical examples, why passing univariate statistical tests is not sufficient evidence that a generator is fit for use in a Monte Carlo study.
7. Explain the practical numerical issues (overflow, precision) that arise when implementing a congruential generator with a large modulus, and describe how Schrage's algorithm addresses them.
8. Discuss the criteria a statistician should apply when selecting a random number generator for a large-scale simulation study, relating each criterion back to the theory developed in this unit.
9. Explain, using the full-cycle moment calculation of Section 6, why full period is necessary but not sufficient for a congruential generator to be considered "good."
10. Trace the historical development of pseudo-random number generation from von Neumann's middle-square method through Lehmer's multiplicative congruential generator to the design principles embodied in the Hull-Dobell theorem.

Programming Exercises

1. Implement `generate_lcg()` from first principles (already provided in this unit) and use it to reproduce the twenty-step table of Example 1 exactly.
2. Write an R function that, given a , c , and m , empirically determines the observed period of the generator from a fixed seed by iterating until the state repeats, and compare the result to the prediction of the Hull-Dobell theorem.
3. Implement `check_hull_dobell()` from first principles (already provided) and use it to test at least five different (a, c, m) combinations of your own choosing, tabulating the results.
4. Reproduce the RANDU lattice-structure plot of this unit using a different pair of small integer coefficients in the planar combination, and comment on whether the banding is still visible.
5. Implement the Park-Miller minimal standard generator ($a = 16807$, $m = 2^{31} - 1$, $c = 0$) and verify empirically, for a sample of size 100,000, that its sample mean and variance are close to the theoretical $\text{Uniform}(0, 1)$ values.
6. Write an R function that searches, for a fixed prime modulus m between 10 and 100, for all primitive roots modulo m , and verify your answer for $m = 31$ against Example 3 of this unit.
7. Using `system.time()`, benchmark your `generate_lcg()` function against R's built-in `runif()` for sample sizes of 10^4 , 10^5 , and 10^6 , and discuss the results.
8. Write a function that implements Schrage's algorithm for computing $(aX) \bmod m$ without intermediate overflow, and verify it against direct computation (using R's arbitrary-precision handling via `as.numeric()`, since R's doubles safely represent integers well beyond the moduli used in this unit) for a range of test inputs.
9. Simulate a mixed congruential generator with a modulus that is a power of two, extract only the lowest bit of each output, and demonstrate empirically that the resulting bit sequence has a much shorter period than the full generator.
10. Write an R function that takes a candidate multiplier, increment, and modulus, verifies the Hull-Dobell conditions, and, if they hold, generates and returns a full period's worth of uniform values, raising an informative error otherwise.

Mini Projects

1. **A parameter atlas for mixed congruential generators.** Using `check_hull_dobell()`, systematically search moduli $m \in \{2^{10}, 2^{16}, 10^4, 10^5\}$ for multipliers a that (i) satisfy the Hull-Dobell conditions and (ii) are not trivially close to 1 (mod m) (say, within $0.001m$). For each modulus, report the number of valid multipliers found, and, for one representative choice per modulus, produce a lag-1 scatterplot analogous to those in this unit. Summarize which moduli offer the richest set of well-behaved multipliers.
2. **Reproducing the RANDU controversy.** Implement RANDU and at least one alternative multiplicative congruential generator with a prime modulus of comparable size (e.g. $m = 2^{31} - 1$). For both

generators, produce three-way lag plots analogous to the two- dimensional plots in this unit (consider projecting the third dimension using colour or facets), run the diagnostic identity of Equation (1.7) numerically to detect planar structure, and write a short report explaining, in your own words, why RANDU's flaw remained undetected for years despite its widespread use.

Difficulty labels. Multiple choice, short answer: **Easy**. Long answer questions 3-7, programming exercises 1-6: **Medium**. Long answer questions 1, 2, 8-10, programming exercises 7-10: **Medium to Advanced**. Mini projects: **Advanced**.

References

- Hull, T. E., & Dobell, A. R. (1962). Random number generators. *SIAM Review*, 4(3), 230-254.
- Knuth, D. E. (1997). *The art of computer programming, Volume 2: Seminumerical algorithms* (3rd ed.). Addison-Wesley.
- L'Ecuyer, P. (1999). Tables of linear congruential generators of different sizes and good lattice structure. *Mathematics of Computation*, 68(225), 249-260.
- Marsaglia, G. (1968). Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, 61(1), 25-28.
- Park, S. K., & Miller, K. W. (1988). Random number generators: Good ones are hard to find. *Communications of the ACM*, 31(10), 1192-1201.
- Ripley, B. D. (1987). *Stochastic simulation*. Wiley.
- Rizzo, M. L. (2019). *Statistical computing with R* (2nd ed.). CRC Press.
- Ross, S. M. (2020). *Simulation* (6th ed.). Academic Press.